My admin module scope

- Admin login page.
- Admin authentication using Spring Security or the chosen security approach.
- Role-based access control so only ROLE_ADMIN can open admin pages.
- Admin dashboard page.
- Admin user management page.
- View all users in a table.
- Promote or demote roles such as customer/admin.
- Activate/deactivate users.
- Search and filter users by username, email, or role.
- Delete user action if allowed by your design.
- Dashboard stats cards like total users, active users, recent registrations.
- Logout feature.
- Error/success messages for login and admin actions.
- Admin-only navigation links and protected routes.
- MySQL + JPA persistence for admin user data.
- Thymeleaf admin templates for login, dashboard, and user management.
- Validation for admin forms.
- Pagination for long user lists.
- Audit-style timestamps like created/updated date if you want a cleaner admin panel.

What your website is about

Sakura Saki is a beauty salon management system where customers, staff, services, appointments, reviews, and admin management are part of one overall platform.
The website is meant to digitize salon operations such as account handling, service management, staff scheduling, appointment booking, and feedback collection.

Project outline

Our project can be explained in these modules:

- Customer and Authentication - register, login, profile, customer account management.
- Service and Package Management - salon services, package offers, pricing, duration.
- Staff and Beautician Management - staff profiles, specialties, availability.
- Appointment Booking - create bookings, view schedules, reschedule, cancel.
- Admin Dashboard and Reports - manage users and view summary statistics.
- Reviews and Post-Visit Flow - ratings, comments, moderation after appointments.

What I should implement

I should implement:

- Admin login.
- Admin dashboard.
- Admin user CRUD / user management.
- Role control.
- Search/filter/pagination.
- Logout and security rules.
- MySQL/JPA storage for admin-related user data.

What I should not do

Do not implement these unless my teammates have not done them and my lecturer says you must:

- Customer booking flow.
- Service/package screens.
- Staff management screens.
- Reviews module.
- Full appointment business logic.

Simple one-line summary

My part is: admin authentication, admin dashboard, user management, role control, and admin-side reporting/search in Sakura Saki.

Coding languages

- Java - for Spring Boot backend, controllers, services, entities, and security logic.
- HTML - for Thymeleaf templates such as login, register, dashboard, and page layouts.
- CSS - for page styling and UI design.
- JavaScript - for frontend interactivity, including the 3D landing page and UI effects.
- SQL - for database creation, queries, and admin/user data handling.

Database type

- MySQL - the project uses a relational SQL database for storing users and app data.
- JPA/Hibernate - used as the ORM layer between Java classes and MySQL tables.

Technologies

- Spring Boot - main backend framework.
- Spring Security - authentication, login, logout, and role-based access control.
- Spring Data JPA - repository and CRUD database access.
- Thymeleaf - server-side HTML template engine.
- Maven - dependency management and project build tool.
- IntelliJ IDEA - development environment you are using.
- Bootstrap - used for responsive UI styling in the pages.
- Three.js - used for the 3D Sakura landing page effect.

In short

Our project stack is:

Java + Spring Boot + Spring Security + Spring Data JPA/Hibernate + MySQL + Thymeleaf + HTML/CSS/JavaScript + Bootstrap + Three.js + Maven.